Networth Matrimony - Project Status Report

Report Date: October 23, 2026 | **Branch:** main (2e6cf9a) | **Repository:** D:\matrimony

Executive Summary

The platform is feature-complete end to end. Phases 0-8 deliver the backend: authentication (with OTP delivery over MSG91), profile and discovery, payments and unlocks (including the section-15 pre-expiry reminder), trust and safety (including the report moderation queue), identity verification, notifications (with push delivery over Firebase), and the admin API with RBAC, audit logging, enrichment tooling, TOTP MFA, the section-30 moderation queue, and the Next.js admin dashboard. Phase 9 delivers the React Native member app: OTP sign-in and registration, discovery with temporary filters and sort, member previews and Razorpay checkout, the account setup fee, profile and photo management, biodata import, visibility preferences, notifications, unlocks, and the feature-flagged For You feed. All gates are green with 888 automated tests passing.

Key Metrics

Metric	Value
Automated Tests	888 (582 API + 29 Admin + 257 Shared + 20 Mobile)
Test Suites	57 (38 API + 4 Admin + 12 Shared + 3 Mobile)
Admin Permission Keys	21
Current Commit	2e6cf9a
Status	All Gates Green

Implementation Status by Phase

Phase	Module	Status	Key Deliverables
0	Foundations	Complete	Schema, config, health, security, monorepo
1	Identity & Auth	Complete	OTP, JWT, rotating sessions, lockout
2	Profile & Master Data	Complete	CRUD, photos, master lists
3	Discovery	Complete	Two-way visibility, filters, pagination
4	Payments & Unlocks	Complete	Razorpay, webhooks, idempotency, expiry
5	Trust & Safety	Complete	Blocking, reporting, account deletion with retention-then-anonymise, report moderation queue
6	KYC/Verification	Complete	Contracts, service, controller, module
7	Notifications	Complete	Shared contracts, API module, list endpoint, unlock pre-expiry reminder, Firebase push delivery
8	Admin API	Complete	RBAC, audit, MFA, enrichment, moderation queue, Next.js dashboard
9	Mobile App	Complete	Expo SDK 57 app: auth, discovery, checkout, setup fee, profile, notifications, For You

Recent Accomplishments

Biodata Import (Member App + API)

- A member can upload a biodata (a typed/exported PDF or a plain text file) and have the recognised fields proposed for the profile form, instead of retyping every detail

- Extraction is a deterministic text parser (no OCR/LLM provider is configured) and runs server-side, so the client ships no parser; `packages/shared/src/biodata-parse.ts` is pure and unit-tested, and `apps/api/src/profile/pdf-text.ts` extracts PDF text on `node:zlib` alone (no new API dependency)
- Same three-step upload shape as photos: the API mints a presigned PUT, the client sends the bytes straight to storage, then posts the object key to `POST /profile/biodata/parse`
- The API reads the document itself and accepts only a key under the caller's own `biodata/` prefix, so it cannot be pointed at another profile's upload or an arbitrary URL; the response is a proposal, never a save
- Education, profession and community are matched against the master lists for a value id; an unmatched one is reported as still needed rather than offered as fillable text
- The member app shows each extracted value with its confidence for review and confirm, the user unticks what is wrong, and the save goes through the ordinary PATCH with the same validators

Phase 9 - Member App (Expo + React Native)

- `apps/mobile` (`@matrimony/mobile`): Expo SDK 57, React Native 0.86, Expo Router, strict TypeScript, consuming `@matrimony/shared` contracts; typed API client validates every request body and response with Zod
- Auth and onboarding: phone OTP request/verify, registration, rotating access/refresh tokens in `expo-secure-store` with transparent refresh on 401, `useRequireAuth` gate, and profile creation with completeness prompts
- Discovery: cursor-paginated infinite feed of locked cards, temporary filters (age, height, city, state, education, profession, verified photos) and sort applied per request, plus a filter/sort bottom sheet; the API now parses nested filter query objects
- Member previews and payments: locked/unlocked/expired states, Razorpay checkout in a WebView with the order `key_id` , status polling against `GET /payments/:id` , and unlock expiry shown in the matches tab
- Account setup fee: verified accounts move to `AWAITING_SETUP_FEE` ; the home feed gates until payment, with a dedicated screen showing the Rs 15.00 + 18% GST = Rs 17.70 breakdown and a one-time order
- Profile: own profile with completeness, field editing, direct-to-storage photo upload with primary/delete management, biodata import, and a visibility toggle
- Settings: net-worth discover/visible-to preference editor, sign out, and OTP-confirmed account deletion
- Notifications: in-app list with unread badges and push registration through `expo-notifications` -> `POST /devices` ; dead tokens are pruned server-side
- For You: curated interest picker, recommendation cards ranked by shared-interest overlap with the common tags shown, a fifth tab, and a graceful "coming soon" state while `FEATURE_AI_RECOMMENDATIONS` is off
- Test suite: Jest via the `jest-expo` preset runs the member app's logic (feed view model, interest helpers) and joins the root `npm run verify` gate; CI also bundles the app (`expo export --platform web`) so a broken route or import fails before a device build

Unlock Pre-Expiry Reminder (Section 15 / C3)

- `UNLOCK_EXPIRY_REMINDER_HOURS` was configured but unused; a sweep now records one notification per unlock about two hours before it expires
- `ContactUnlock.reminderSentAt` marks a reminder sent and the claim is a conditional update, so a restart, redeploy or second instance cannot notify a buyer twice
- `UnlockExpiryReminder` follows the existing sweeper/ `ClockDriftGuard` timer shape (5-minute interval, `unref` , boot sweep, failures logged not thrown)
- Channel is `PUSH` ; the row also remains in the in-app list, which is not filtered by channel

Push Delivery over Firebase (Section 42, resolves GAP-12's push half)

- `PushGateway` seam bound to `FcmPushGateway` in `PushModule` : FCM HTTP v1 with a service-account assertion (RS256 JWT exchanged for a cached OAuth2 token) on `node:crypto` ??? no legacy server key, no `firebase-admin` ; fails closed when unconfigured
- `DeviceToken` model + `DevicePlatform` enum; `NotificationStatus` gains `SENDING` for the in-flight claim; migration `20261014140000_device_tokens_push`
- `POST/GET /devices` and `DELETE /devices/:token` under `JwtAuthGuard` ; registration upserts on the token and reassigns its owner, and the token is never echoed back
- `PushDispatchService` claims a row `PENDING` -> `SENDING` with a conditional update (at-least-once, never twice-per-worker), requeues a stale `SENDING` row, prunes a token FCM reports `UNREGISTERED` , and marks `SENT` / `FAILED` ; `PushDispatchSweeper` runs on a 60-second timer
- A `PUSH` row for a member with no device stays `PENDING` and is delivered once one is registered

SMS OTP Delivery (MSG91, resolves GAP-12)

- `SmsGateway` seam (abstract class) with a `Msg91SmsGateway` on MSG91's Flow API, bound like the payment gateway so tests substitute it without the network; a small `SmsModule` keeps the module graph acyclic
- `AuthController.requestOtp` sends the code via MSG91 in production and fails closed (401) when MSG91 is unconfigured; outside production it still echoes `dev_code` so the flow stays testable without a provider account
- Flow API rather than MSG91's SendOTP API, because the platform owns OTP state (hashed, attempt-counted, expired, consumed at later steps); the code travels as a template variable and MSG91 only delivers it
- New config `SMS_TEMPLATE_ID` / `SMS_OTP_TEMPLATE_VAR` / `SMS_COUNTRY_CODE`; 9 gateway tests

Admin MFA (TOTP)

- Hand-rolled RFC 6238 TOTP and AES-256-GCM secret-at-rest encryption, no new dependency
- Opt-in self-service enrollment; the secret is persisted only after a matching code
- Login for an MFA-enabled admin returns a signed challenge; only the verify endpoint issues a session
- Wrong codes share the password lockout counter; disable requires a current code; audited admin:manage reset for lost devices

Account Retention Purge (Section 21)

- `deleteAccount` set `purgeAfter` but nothing read it; a new `AccountRetentionSweeper` now anonymises an account once its retention window closes
- `AccountDeletionService.purgeDue` overwrites the mobile with a per-account tombstone (`purged:<id>`, so the real number can register again), clears email, blanks the profile's PII and demographics, and deletes photos and interests
- Payment and ContactUnlock records are kept, because section 21 retains the accounting for a person who is erased; the `User` / `Profile` rows survive anonymised, so their foreign keys stay intact
- Idempotent by construction: the query matches only `isAnonymised: false` rows past `purgeAfter`, so a re-run or a second instance is a no-op

Report Moderation Queue (Section 30)

- GET /admin/reports (status filter + cursor pages), GET /admin/reports/:id, POST /admin/reports/:id/review, all behind report:review
- Resolution is terminal and pairs action with status; notes required to action
- SUSPEND suspends the reported account as its only side effect, guarded against reviving a DELETED account
- Every review writes a report:review audit row

AI Recommendations (Backend + For You tab)

- Curated interest tags with GET/PUT /profile/interests
- Feature-flagged For You feed (GET /recommendations) ranked by shared-interest overlap
- Reuses the existing discovery eligibility and two-way acceptance scope
- The member app exposes it as a fifth tab with an interest picker and a flag-off state

Admin Dashboard (Next.js)

- `apps/admin` (`@matrimony/admin`): Next.js 16 App Router + React 19, talking only to the admin API
- Backend-for-frontend auth: the token pair lives only in httpOnly cookies; an edge proxy guards every route and refreshes the access token before it expires
- Password-then-TOTP login; session route handlers re-validate every API response with the shared Zod contracts
- Pages: overview, the moderation queue (status filter + detail review form), member list, administrator/role list
- Build pinned to webpack (the Next 16 Turbopack default panics writing this app's route chunks on Windows)

Quality Assurance

- Tests: 888 passed (582 API + 29 Admin + 257 Shared + 20 Mobile)
- Typecheck: PASS (workspaces + tools, including the seed and the mobile app)
- Lint: PASS
- Build: PASS (shared + API + admin)
- Full verification: PASS

Next Steps

- Device-level QA of the Expo app (dev build) and the pending store identifiers (`bundleIdentifier` / Android `package`)
- Live provider checks: a real Razorpay payment and webhook, a real FCM push, and a real MSG91 SMS; the webhook secret, sender/template/DLT ids and email provider key are still unset
- Flip `FEATURE_AI_RECOMMENDATIONS` on after a gradual rollout decision; the For You surface is already shipped behind the flag
- Email provider is still open (the remaining half of GAP-12; SMS and push are resolved)
- Client clarifications still open: GAP-11 privacy wording (R2 India is best-effort) and the email half of GAP-12. GAP-2 (net-worth boundaries) is closed: a value exactly on a boundary goes into the higher band, which is what the build already does